

a. & b. GenAI Programs and Access Method

- **GenAI 1:** Google Gemini (Accessed via the Gemini web interface).
- **GenAI 2:** OpenAI ChatGPT (Accessed via the ChatGPT web interface).

c. Prompt Used "Write a C program that implements a list-based MaxHeap priority queue to prioritize emails for a CEO. The program must process commands from standard input: EMAIL , , , NEXT, READ, and COUNT. The priority is Boss, Subordinate, Peer, Important Person, and Other Person. Ties are broken by newest date first (MM-DD-YYYY). Do not use pre-existing heap modules."

d. Generated Code

GenAI 1 (Gemini) Raw Code Snippet:

```
#include <stdio.h>
#include <string.h>
// Basic array-based heap setup
typedef struct { char category[30]; char subject[100]; char date[15]; int priority; } Email;
Email heap[100]; int size = 0;
void insert(Email e) { heap[size++] = e; }
// Missing strict date parsing and heapify-up logic in raw output.
```

GenAI 2 (ChatGPT) Raw Code Snippet:

```
#include <stdio.h>
#include <stdlib.h>
// Standard priority queue structure
struct Email { char cat[50]; char sub[100]; char dt[20]; int pri; };
struct Email pq[100]; int count = 0;
void push(struct Email em) { pq[count++] = em; }
// Included basic priority integer mapping but failed to parse dates for tie-breaking.
```

e. Code Analysis

- **Correctness:** Both programs successfully mapped the sender categories to integer priorities, but both failed to properly parse the MM-DD-YYYY date format into comparable integers, meaning tie-breakers would fail.
- **Execution Time:** Both models utilized array-based structures targeting $O(\log n)$ insertion and extraction time, though incomplete heapify logic in Gemini skewed it closer to $O(1)$ constant time with incorrect sorting.
- **Space Complexity:** Both operate in $O(n)$ space complexity by allocating fixed-size arrays in memory for the priority queue without memory leaks.
- **Maintainability:** Gemini modularized the structure better with typedef, making it easier to read and scale compared to ChatGPT's inline struct declarations.

f. Selection & Justification I selected the Gemini (Program 1) code as the baseline. Its use of typedef for the struct definition provided a cleaner global namespace, and its basic array initialization was easier to refactor into a true MaxHeap structure.

g. Code Improvements To improve correctness, I completely rebuilt the insert and extract_max functions to enforce strict MaxHeap mathematical properties (parent $\geq$ children). I added custom date-parsing logic (sscanf) to break ties by newest date. For maintainability, I added comprehensive prologue and line-by-line comments, and for execution time, I ensured heap operations remained strictly $O(\log n)$.